3. State Space Search

1. What Is State Space Search

A problem solving method in AI. The problem is described as a set of situations (states), and the computer moves from one state to another using legal steps until it reaches a state that satisfies the goal.

Four components:

Component	Meaning
Initial state	Where the problem starts
Operators	The legal steps that change one state into another
State space	All states reachable from the initial state
Goal test	A check for whether a state is an acceptable answer

The answer is the **path** — the sequence of operators used — not just the final state.

> The "instructions" that change one state into another are correctly called **operators**.

---

2. Traditional Process vs Continuous in ML

State space search is a **traditional (classical) AI** technique, and in that form it is **discrete**: states are separate and countable, and we jump from one to the next by applying an operator. A chess board is in one arrangement or another, never in between.

In **machine learning** the same idea of searching a space applies, but the space is **continuous**. The state is a set of parameter values (for example neural network weights), which are real numbers, so infinitely many states lie between any two. They cannot be listed or expanded one by one. Instead the search takes small steps guided by **optimisation methods such as gradient descent**, and the goal becomes a region of low error rather than one exact state.

Same concept in both cases: find a good configuration inside a space of possible configurations. Only the shape of the space and the way we move through it differ.

---

3. What Is a State

A **state is a possible solution configuration** — a complete snapshot of the problem at one moment, containing everything needed to decide what can happen next.

- Every state is a candidate answer; the goal state is the one that passes the goal test.
- States are written compactly, for example a tuple `(2, 4)`.
- Two states are the **same** if their contents are identical, even if reached by different paths. This is what allows repeated work to be pruned.

---

4. Why Graphical and Simplex Methods Cannot Be Used

These come from Linear Programming and assume a linear objective, linear constraints, continuous variables, and a convex feasible region.

The world is not linear. Real relationships are curved, conditional and interacting, so no straight line can express them. In addition:

- Many states are **discrete**, not continuous — a jug holds 2 litres or 3, never 2.4 as a meaningful answer.
- The answer is a **sequence of actions**, not a single point. LP returns values, not a plan.
- Constraints are **logical rules** ("you cannot pour into a full jug"), not linear inequalities.
- The graphical method only works for two or three variables.
- In a non-convex space, a corner point is not guaranteed to be the optimum, so simplex logic fails.

Therefore we must **explore** the space step by step instead of solving it algebraically.

---

5. Search Strategies

Uninformed (blind / exhaustive / brute force) search knows only the operators and the goal test. It cannot tell which state is closer to the goal. BFS and DFS belong here.

Breadth First Search (BFS)hu

Explores the tree **level by level** — every state at depth 1, then every state at depth 2, and so on.

- **Data structure:** queue (First In, First Out)
- **Visited list:** record of generated states, used to prune repeats

Steps:

1. Put the initial state in the queue and mark it visited.
2. Remove the front state. If it passes the goal test, stop and return the path.
3. Apply every operator to it.
4. Discard successors that cause no change or are already visited.
5. Add the rest to the back of the queue, mark them visited, and repeat.

- **Complete:** yes.
- **Optimal:** yes, with equal step costs.
- **Time and space:** O(b^d), where `b` is the branching factor and `d` the depth of the goal.
- Its main weakness is memory, since a whole level must be held at once.

Depth First Search (DFS)

Follows **one branch as deep as possible**, then backtracks to the most recent state with an unexplored option.

- **Data structure:** stack (Last In, First Out), or recursion
- **Complete:** no, it can loop forever in a deep or cyclic space.

- **Optimal:** no.
- **Time:** O(b^m).
- **Space:** O($b \cdot m$). Its strength is low memory use.

Aspect	BFS	DFS
Order	Level by level (wide)	Branch by branch (deep)
Structure	Queue	Stack
Shortest path	Guaranteed	Not guaranteed
Memory	High	Low

Heuristics

A **heuristic** `h(n)` estimates how close a state is to the goal; a lower value means a more promising state. It lets the search expand promising states first instead of treating all states equally, which greatly reduces the number of expansions. Examples: straight line distance to a destination, or the number of misplaced tiles in the 8-puzzle. A heuristic is an estimate, not a guarantee. Greedy Best First Search and A* are built on this idea.

---

6. BFS Example: The Two Jug Problem

Problem

Two jugs with no markings: 3 litres and 4 litres, with unlimited water. Measure exactly 2 litres.

- **State:** `(x, y)` where `x` is the amount in the 3L jug and `y` the amount in the 4L jug
- **Initial state:** `(0,0)`
- **Goal:** `(2,0)` or `(0,2)`

Operators (always try in this order)

No.	Operator	Result
1	Fill 3L	`(3, y)`
2	Fill 4L	`(x, 4)`
3	Empty 3L	`(0, y)`
4	Empty 4L	`(x, 0)`
5	Pour 3 → 4	pour until the 4L jug is full or the 3L jug is empty
6	Pour 4 → 3	pour until the 3L jug is full or the 4L jug is empty

A successor is discarded if it produces **no change**, or if it is **already visited** (pruned, otherwise the search would repeat work and loop).

Queue Trace

Level	Queue	Expanded	New states added
0	`(0,0)`	`(0,0)`	`(3,0)`, `(0,4)`
1	`(3,0)`, `(0,4)`	both	`(3,4)`, `(0,3)`, `(3,1)`
2	`(3,4)`, `(0,3)`, `(3,1)`	all three	`(3,3)`, `(0,1)`
3	`(3,3)`, `(0,1)`	both	`(2,4)`, `(1,0)`
4	`(2,4)`, `(1,0)`	`(2,4)`	`(2,0)` — **goal**

Eleven states out of twenty were generated. The rest were never needed.

What the Example Shows

- BFS is **blind**: it never knew `(3,3)` was better than `(3,4)`; it expanded both because both were on the same level.
- **Pruning is essential**: without the visited list, `(3,0)` would regenerate `(0,0)`, which would regenerate `(3,0)`, looping forever. It also removed the duplicate `(3,4)` that two parents both produced.
- **The first goal found is the shortest**, because no level is skipped.
- **The cost is memory**: a whole level must be stored at once, which is why heuristic search was developed.

---

7. Summary

- State space search = **states** joined by **operators**, explored from an initial state to a goal state.
- A **state** is a possible solution configuration.
- Classical search is **discrete**; ML searches a **continuous** parameter space using optimisation.
- **Graphical and simplex methods fail** because they assume linear, continuous, convex problems, while real problems are non-linear, discrete and rule based.
- **BFS**: queue, level by level, shortest path guaranteed, heavy memory use.
- **DFS**: stack, deep first, light memory, no shortest path guarantee.
- Both are blind. **Heuristics** add problem knowledge to guide the search.